DAILYOPS

BEYOND THE FIREWALL

The Reality of Modern IT Operations

Ba-Ndouwe Steve

THE OPERATOR'S LIBRARY — BOOK 1

The Operator’s Library — Book 1

Contents

1

Beyond the Firewall

1.1 The Reality of Modern IT Operations

Ba-Ndouwe Steve

The Operator's Library — Book 1

2

Chapter 1 — Beyond the Firewall

2.1 Hook

It was a Tuesday morning when the call came. A manufacturing company had been experiencing intermittent outages for weeks. Nothing dramatic — no explosions, no ransomware, no front-page news. Just a slow, grinding degradation that made people late for meetings, delayed shipments, and forced the IT team to work late every night.

They had a firewall. A good one. Enterprise-grade. Properly configured by a team of certified engineers. It sat at the edge of their network like a silent guard, filtering traffic, blocking threats, logging every packet.

And it was working perfectly.

The problem was not the firewall. The problem was everything else.

I walked into their office with a simple question: “What changed recently?” The answer came quickly: “Nothing. That’s why we don’t understand.”

They showed me their monitoring dashboard. Green across the board. CPU usage normal. Memory fine. Network traffic within expected ranges. The firewall was processing packets without errors. According to every tool they had, the system was healthy.

But their users were complaining. Orders were stuck. A critical internal application was responding so slowly that some employees had given up using it.

I asked to see the application flow. Not the firewall logs — the path a single transaction took from the moment a user clicked “Submit” to the moment the database confirmed it.

They couldn’t show me.

Because nobody had ever mapped it.

2.2 The Bad Belief

We believe that a secure perimeter equals a healthy system. We believe that if nothing gets in from outside, nothing can go wrong inside. This is the first illusion the modern operator must unlearn. Security is a property of the boundary. Reliability is a property of the whole. They are not the same thing, and one does not guarantee the other.

2.3 Principle

The firewall marks the visible edge of your system. Everything that determines whether the system survives, performs, and adapts happens behind it — in the people, the dependencies, the undocumented exceptions, and the decisions made under pressure. Understanding the system requires looking past the firewall.

2.4 Evidence

Consider the manufacturing company's case in detail. The firewall — a standard stateful inspection device — blocked 12,847 malicious packets in March 2023. The security dashboard showed a 99.97% success rate. The CISO reported this to the board as evidence of operational health.

Behind the firewall, the architecture was simpler than anyone admitted. A single PostgreSQL instance served both the warehouse system and the billing service. There was no read replica. There was no failover tested in production — the rollback plan existed as a 12-page Word document, written in 2019, never executed. The exception rule allowing a subcontractor's IP through the firewall — added in a rush in 2021 — had no expiration date.

When the database connection pool saturated, neither service could recover gracefully. There was no circuit breaker. The firewall saw no attack traffic. The monitoring dashboards — focused on CPU, memory, and firewall hits — showed green. The users experienced complete failure.

The firewall was working. The business wasn't.

The most dangerous dependency is not a server. It is the assumption that the firewall defines the system's health.

2.5 Analysis

Why does this persist? Three mechanisms work together.

First, visibility bias. Metrics that are easy to collect dominate the conversation. Firewall logs are structured, timestamped, and easy to visualize. Internal dependency graphs are messy, often undocumented, and resist simple dashboards. The organization measures what it can see, and gradually confuses visibility with control.

Second, organizational distance. The team that manages the firewall is rarely the team that operates the warehouse system. Their success metrics are different. The firewall team is rewarded for zero breaches. The operations team is rewarded for uptime — but uptime is measured at the service level, not at the dependency level. The gap between these metrics is where failures grow silently.

Third, the pressure of urgency. When a subcontractor needs access, the firewall exception is added quickly. There is no mechanism forcing expiration. There is no review tied to the exception. The decision is rational at the time — the business needs to move. But the mechanism for reversal is missing. The exception becomes permanent not by design, but by neglect.

This is the pattern that defines the invisible system: rational decisions, missing mechanisms, silent accumulation of risk.

2.6 Application

If you operate or oversee any system with a firewall — which is to say, any modern IT system — start with three actions.

One: map the dependencies that the firewall cannot see. Not the public-facing services, but the internal connections that determine whether those services work. Which database serves more than one critical function? Which API has no documented consumer list? These are the real architecture.

Two: separate security metrics from reliability metrics. A green firewall report tells you nothing about recovery time, dependency health, or organizational readiness. Build a separate view — not a replacement, an addition — that tracks what matters to the user, not to the perimeter.

Three: establish expiration mechanisms for every exception. A temporary firewall rule without an expiration date is not temporary. It is deferred maintenance. The same applies to access grants, cloud resources, and manual workarounds. If it has no expiration, it has no accountability.

The firewall does its job. Your job is to understand what happens behind it.

2.7 Memorable Phrase

> The firewall was working. The business wasn't.

2.8 FIELD NOTE

Context: A manufacturing company experienced intermittent outages over several weeks. The firewall and monitoring dashboards showed no errors.
What We Expected: Green metrics meant a healthy system.
What Happened: A critical order-processing application became increasingly slow. Users could not complete transactions. No breach occurred.
Why It Happened: The application pointed to a database that had been moved during an infrastructure upgrade. The firewall and monitoring were not designed to detect this internal dependency failure.
What It Taught Us: A secure perimeter does not guarantee a functioning business. The real health of the system is invisible to the firewall.

2.9 The Principle: Look Beyond the Component

The principle of this chapter is simple but profound: do not confuse the health of a component with the health of the system. A firewall can be working perfectly while the business is failing. A server can be running smoothly while an application is broken. A dashboard can stay green while users suffer.

To see clearly, we must learn to look at the connections, not just the boxes. We must ask not only “Is it up?” but also “Does it work?” Not only “Is it secure?” but “Is it reliable?” Not only “What is the

firewall doing?" but "What is the user experiencing?"

This is the first step beyond the firewall: broadening our view from the visible boundary to the entire operational reality.

2.10 Pull Quote

The firewall reveals the boundary. The boundary is not the failure.

2.11 Transition

The firewall reveals the boundary. But the boundary is not the failure. In the next chapter, we examine what happens when an exception — temporary, justified, urgent — crosses the firewall and never leaves: when the temporary rule becomes permanent architecture.

2.12 KEY TAKEAWAYS

> Key Takeaways
>
> - The firewall is a visible boundary, but it is not the whole system. The real risks are hidden inside, invisible to

perimeter monitoring.

- A system's health cannot be measured by checking individual components. It must be measured by the experience of the people who rely on it.
- The real frontier of IT operations is the gap between what we think we know and what is actually happening.
- To operate effectively, we must shift from a component-level view to a systems-level view.
- The first step beyond the firewall is to ask: "Is the business working?" — not just "Is the firewall working?"

3

Chapter 2 — The Temporary Rule That Became Permanent

3.1 Hook

The firewall rule had no expiration date. It was created in a hurry, during a crisis, with a note: “Temporary — remove after fix.” Two years later, it was still there.

3.2 The Discovery

I remember the afternoon clearly. A financial services company asked me to review their firewall configuration as part of a security audit. They were confident. Their perimeter was solid. The firewall was the crown jewel of their defense.

We sat in a small conference room, a projector humming, the firewall rules scrolling on the screen. Hundreds of lines. Most were clean, well-documented, and logically grouped. But then, near the bottom, I saw something that made me pause.

Rule 214. Source: Any. Destination: Any. Port: 22. Allow.

No description. No owner. No expiration. Just a single line that allowed SSH from anywhere to anywhere.

I turned to the network administrator. "What's this rule?"

He frowned, squinted at the screen, and said the words I've heard a hundred times since:

"That? Oh, that's temporary. We opened it for a vendor last year. We'll remove it soon."

I pointed to the creation date. It was twenty-three months old.

He stared at it for a long moment. Then he whispered, "We forgot."

3.3 The Illusion of Temporary

This is not a rare story. It is one of the most common patterns in IT operations. A problem appears. A firewall rule is added to solve it quickly. The team says: "We'll remove it later." But later never comes.

The rule stays. The vendor contract ends, but the access remains. The developer leaves, but the exception lives on. The urgent fix becomes a permanent hole in the perimeter.

3.4 The Bad Belief

We believe that a temporary exception is harmless. We believe that once the crisis passes, the exception will be cleaned up naturally — by memory, by review, or by good intentions. This belief ignores the mechanism of neglect. Without an expiration date, a temporary exception becomes architecture not by decision, but by default.

3.5 Why It Happens

Pressure. When a system is down, or a vendor needs immediate access, there is no time for process. The priority is to restore service. The firewall rule is the fastest solution. It is added in minutes, tested, and works. Everyone is relieved. The crisis is over.

But the crisis leaves a trace. A rule that was born in urgency, without a plan for removal.

Forgetfulness. The human brain is not designed to remember every temporary exception. We move on to the next incident, the next project, the next fire. The rule remains in the configuration, invisible and forgotten.

The absence of an expiration mechanism. Most firewall platforms do not force an expiration date. If you don't set one manually, the rule lives forever. And because no one thinks about it, it stays.

The result is what I call the Permanent Temporary.

3.6 Evidence / Incident

The discovery of Rule 214 led to a broader audit. The company believed their firewall was clean. What we found was a pattern of silent accumulation.

The firewall logs showed zero blocked intrusions in the previous quarter. The security team presented this as evidence of health. Behind the firewall, the architecture was simpler than anyone admitted. A single exception — created for a vendor integration that had ended months earlier — allowed unrestricted SSH access from any source. There was no documentation linking the rule to the vendor contract. There was no expiration. There was no review process in place.

When the audit revealed the rule, the team's first reaction was not concern — it was confusion. They had genuinely forgotten it existed. The mechanism for remembering had never been built. The exception had become invisible not by design, but by neglect.

The firewall saw nothing unusual. The firewall was working. The exception was permanent.

3.7 DAILYOPS CONCEPT

> **PERMANENT TEMPORARY**
> *A temporary exception that becomes permanent operational architecture, without a mechanism for review or removal.*

3.8 The Cost of Forgetting

When I showed the company how many similar rules existed — not just firewall rules, but temporary user accounts, unused VPN profiles, old IAM permissions — they were shocked. They had believed they were secure because their main systems were protected. But the perimeter was full of tiny holes, each one invisible to their monitoring dashboards.

This is the illusion of control. We look at the firewall and see a solid wall. But the wall is made of thousands of rules, each one a potential gap. The dashboard shows green, but the wall is already leaking.

The company had a firewall that was working. The business was not protected from its own exceptions.

3.9 Analysis

The Permanent Temporary is not a security problem alone. It is a symptom of organizational failure. Every exception represents a deviation from the designed state. If that deviation is not tracked, reviewed, or removed, it becomes part of the architecture. And architecture, by definition, is durable.

The mechanism that creates this failure has three parts.

First, urgency overrides process. When a vendor needs access or a service must be restored, the exception is the fastest path. There is no time to design a permanent, secure solution. The exception is rational — at the moment.

Second, the absence of an expiration mechanism means the exception lives by default. If there is no date forcing a review, the exception outlasts the crisis. It survives the departure of the engineer who created it. It survives the end of the vendor contract. It survives the organizational memory.

Third, the lack of documentation connects the exception to nothing. There is no link to the vendor, the project, or the crisis that created it. Without this link, no one can evaluate whether the exception is still needed. It exists in isolation, invisible to everyone except the firewall itself.

This pattern is not rare. It is common. And it is dangerous because it is invisible.

3.10 Principle

Every exception needs an expiration mechanism. Every temporary fix must have a date — a date by which it must be reviewed, renewed, or removed. Without this mechanism, temporary becomes permanent. With it, every exception remains accountable to the organization that created it.

3.11 Application

If you manage or oversee any firewall, any access control list, or any temporary resource, implement three rules immediately.

One: set an expiration date. Every exception must have a date — one week, one month, one quarter — by which it must be reviewed. When the date arrives, the exception does not renew automatically. Someone must justify its continuation.

Two: assign an owner. Every exception must have a named person responsible for it. That person receives the notification when expiration approaches. Without an owner, no one is accountable.

Three: document the reason. Every exception must have a brief explanation: what problem it solves, who requested it, and when it should be removed. Without documentation, no one can evaluate whether the exception is still needed.

These three rules transform temporary fixes from silent risks into managed deviations. They create visibility where there was only

neglect. They turn exceptions into architecture that can be reviewed, not architecture that hides.

3.12 FIELD NOTE

Context: A financial services company conducted a firewall audit. A single SSH exception rule was discovered with no expiration or owner.
What We Expected: Firewall rules would be documented, justified, and regularly reviewed.
What Happened: Rule 214 allowed SSH from any source to any destination. It was 23 months old, labeled temporary, and never removed.
Why It Happened: The rule was added during an urgent vendor integration. No expiration was set. No review process existed. The engineer left, and the rule was forgotten.
What It Taught Us: Every temporary exception needs an expiration mechanism. Without one, urgency becomes architecture.

3.13 Memorable Phrase

A temporary exception without an expiration date is a future security incident.

3.14 Pull Quote

The firewall was working. The exception was permanent. The business was not protected.

3.15 Transition

The firewall reveals the boundary. But the boundary is not the failure. In the next chapter, we go deeper behind the firewall — not just past the rules, but into the dependencies that nobody documented: the hidden architecture that holds the system together, and that can break it without anyone noticing.

3.16 KEY TAKEAWAYS

Key Takeaways

- A temporary exception without an expiration date is a future security incident.
- The Permanent Temporary is created by urgency, forgetfulness, and the absence of review mechanisms.
- Every exception must have an owner, an expiration date, and a documented reason.
- Trust without verification is not an operational control.
- The discipline to remove temporary fixes is what separates mature operations from fragile ones.